

HIGH PERFORMANCE COMPUTING

UNIT 1

Introduction to Parallel Computing



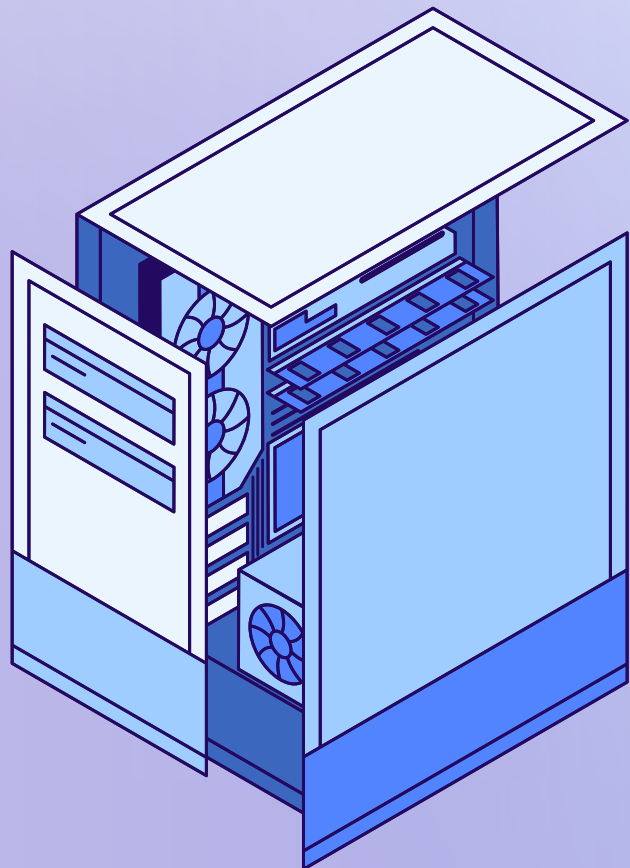
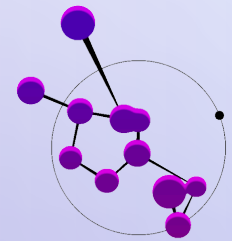
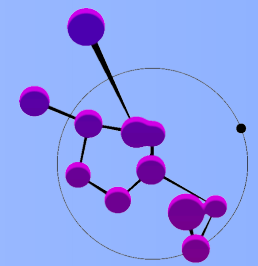
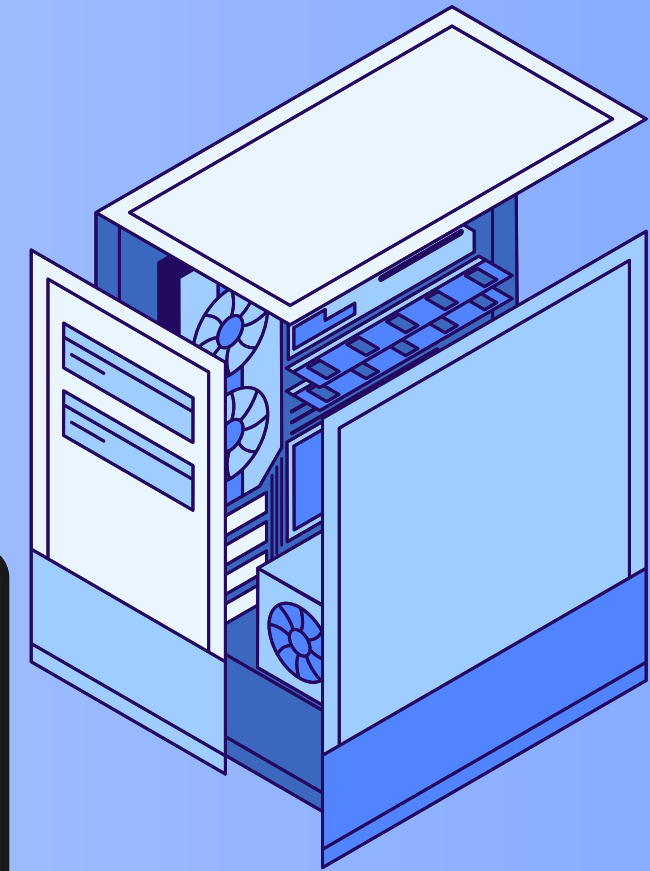
DEFINITION AND SCOPE

What is Parallel Computing?

- Performing multiple computations simultaneously
- Uses multiple processors / cores
- Solves large problems faster than sequential computing

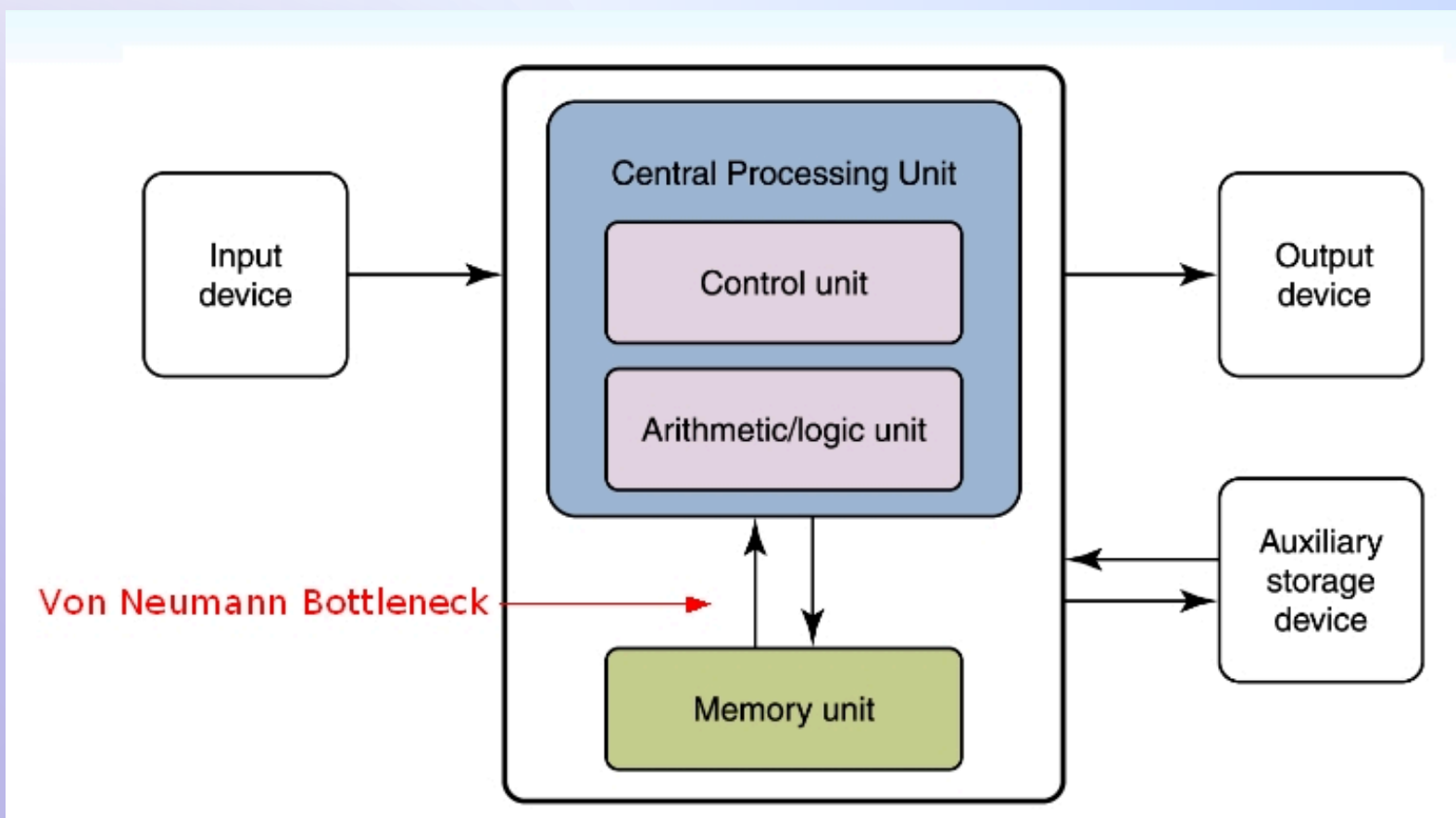
Why Do We Need Parallelism?

- 📈 Increasing data size (Big Data, AI, ML)
- ⚡ Need for high speed & performance
- 💻 Modern multi-core processors
- 🌐 Real-time applications (Weather Forecasting, Online Services)



MODERN PROCESSOR

STORED PROGRAM COMPUTER ARCHITECTURE



Stored Program Concept

- Instructions and data are stored in the same memory
- Program can be modified during execution
- Based on John von Neumann architecture

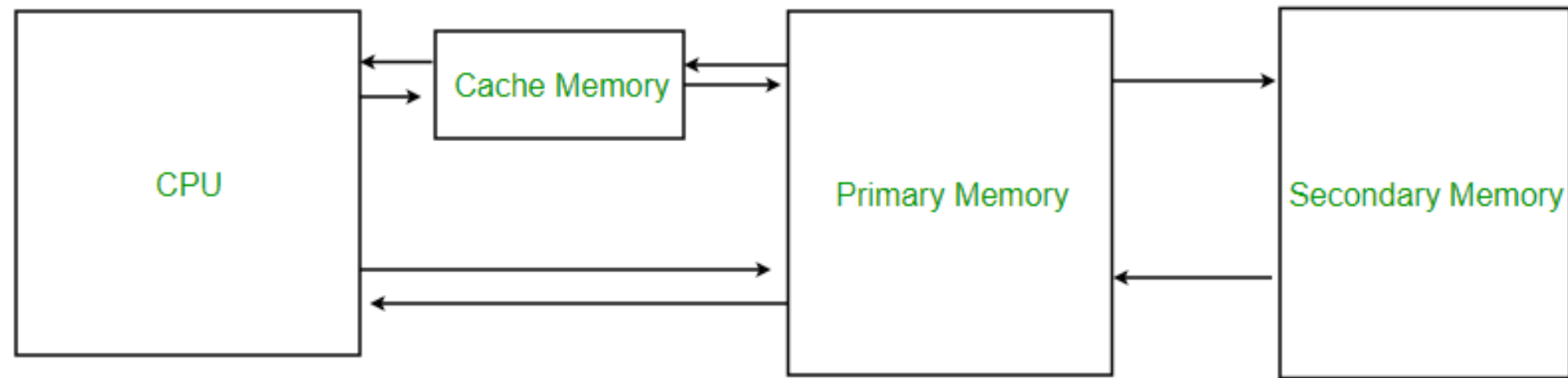
Main Components

- CPU (Central Processing Unit)
 - ALU (Arithmetic Logic Unit)
 - Control Unit
 - Registers
- Memory (RAM)
- Input Devices
- Output Devices

Working Principle

1. Fetch instruction from memory
 2. Decode instruction
 3. Execute instruction
 4. Store result
- (Fetch–Decode–Execute Cycle)

GENERAL-PURPOSE CACHE-BASED MICROPROCESSOR ARCHITECTURE



- Cache memory is placed between CPU and Main Memory
- Cache is small but very fast memory
- Stores frequently used data and instructions
- Reduces access time and improves system performance

Main Components

CPU

- Executes instructions and performs calculations
- Contains Control Unit, ALU, Registers

Cache Memory

- Very fast but small memory
- Stores frequently accessed data
- Provides quick access to CPU

Primary Memory (RAM)

- Temporarily stores programs and data
- Slower than cache but faster than secondary storage

Secondary Memory

- Permanent storage (HDD, SSD, Pen Drive)

Working

- CPU requests data
- Check in Cache → Cache Hit / Cache Miss
- If miss → Data fetched from Main Memory
- Frequently used data stored in cache



PARALLEL PROGRAMMING PLATFORMS

IMPLICIT PARALLELISM



Basic Idea

- Parallelism handled automatically by the system
- Programmer writes sequential code
- Compiler, runtime system, or hardware extracts parallelism

Key Features

- No need for explicit thread or process management
- Automatic task distribution across processors
- Improves performance without complex programming

Examples of Implicit Parallelism

- Compiler-based parallelization
- Automatic vectorization
- Parallel execution in modern multi-core processors

Advantages

- ✓ Easier programming
- ✓ Reduced development complexity
- ✓ Efficient use of multi-core processors



DICHOTOMY OF PARALLEL COMPUTING PLATFORMS

Basic Idea

- Parallel computing platforms are classified into two main categories
- Based on how processors access and share memory

Shared Memory Architecture

- Multiple processors share a common memory space
- Communication through shared variables
- Easier programming model

Examples

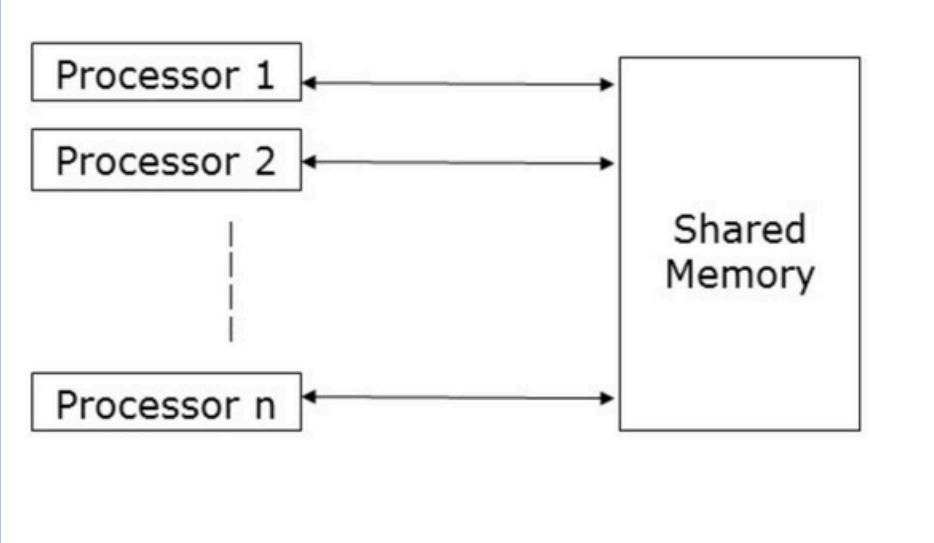
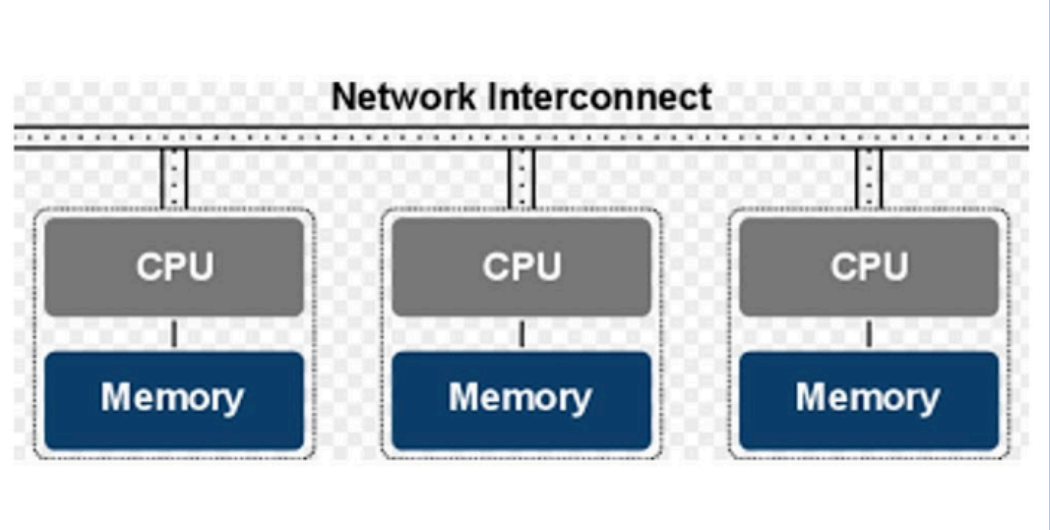
- Multi-core processors
- SMP (Symmetric Multiprocessor)

Distributed Memory Architecture

- Each processor has its own local memory
- Processors communicate through message passing
- Suitable for large-scale systems

Examples

- Computer clusters
- Supercomputers



Summary of Key Differences

Aspect	Shared Memory Architecture	Distributed Memory Architecture
Memory Access	Single shared memory space	Local memory for each processor
Communication	Direct access to shared data	Message passing between processors
Programming Model	Easier, with fewer complexities	More complex, requiring explicit communication
Scalability	Limited scalability due to contention	High scalability with minimal contention
Cache Coherence	Required for shared data consistency	Not applicable as each processor has local memory
Performance Bottlenecks	Contention and synchronization overhead	Network latency and data transfer overhead



PHYSICAL ORGANIZATION OF PARALLEL PLATFORMS

Describes how processors, memory, and communication networks are physically connected in a parallel system.

Main Components

◆ Processors

- Perform computations in parallel
- Multiple CPUs/cores work together

◆ Memory Organization

- Shared Memory – all processors access the same memory
- Distributed Memory – each processor has its own local memory

◆ Interconnection Network

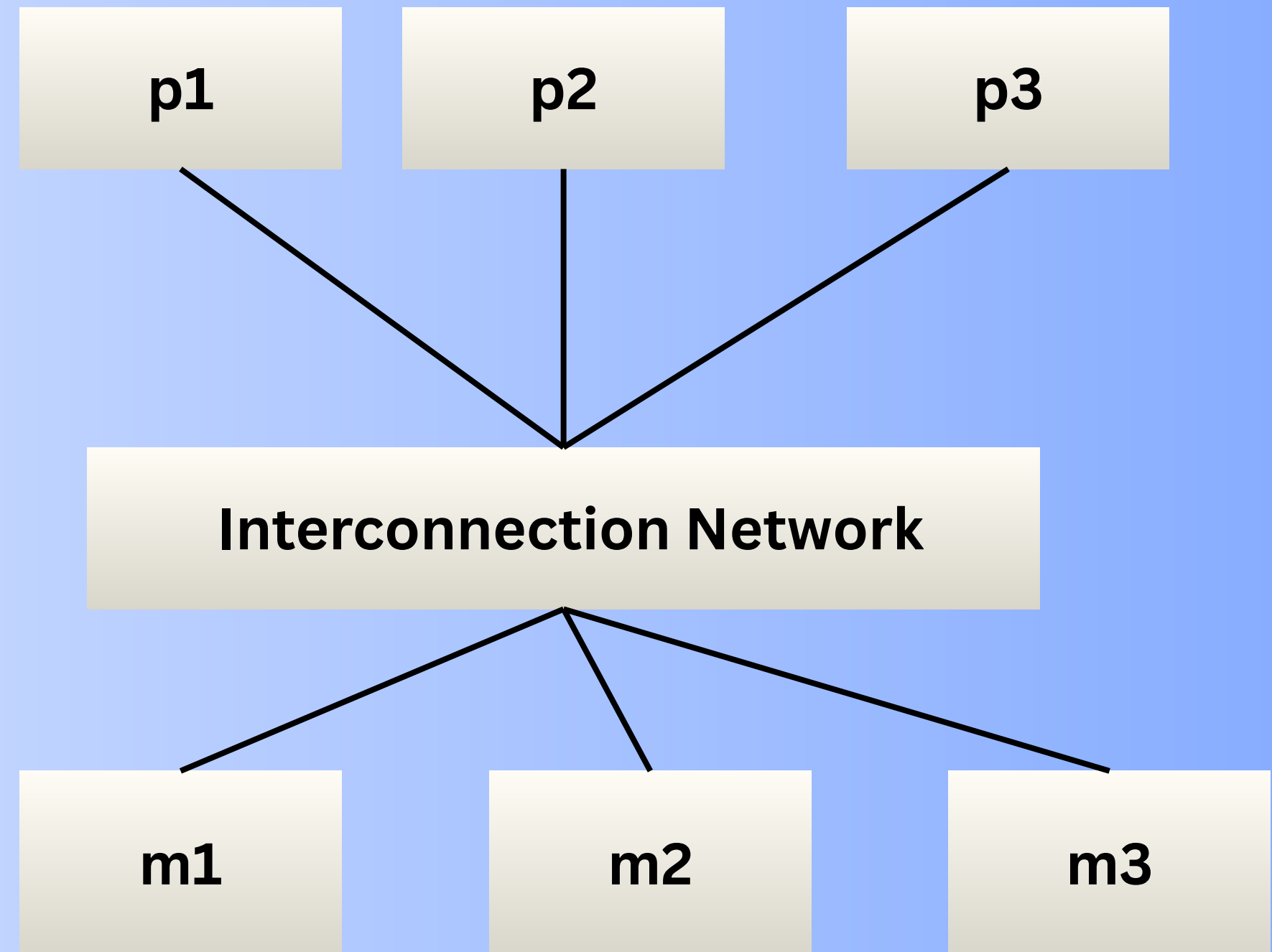
- Connects processors and memory
- Enables data communication between processors

Types of Parallel Platforms

- Multi-core / Shared Memory Systems
- Clusters
- Massively Parallel Processors (MPP)

Goal

- ✓ High performance
- ✓ Faster computation
- ✓ Efficient communication between processors



PRAM MODEL AND INTERCONNECTION NETWORKS

PRAM Model (Parallel Random Access Machine)

- Theoretical model used to design parallel algorithms
- Multiple processors share a common global memory
- Processors work synchronously

- EREW – No simultaneous read or write
- CREW – Multiple processors can read simultaneously
- CRCW – Multiple processors can read and write simultaneously

Model	Read	Write
EREW	Exclusive	Exclusive
CREW	Concurrent	Exclusive
CRCW	Concurrent	Concurrent

Interconnection Networks

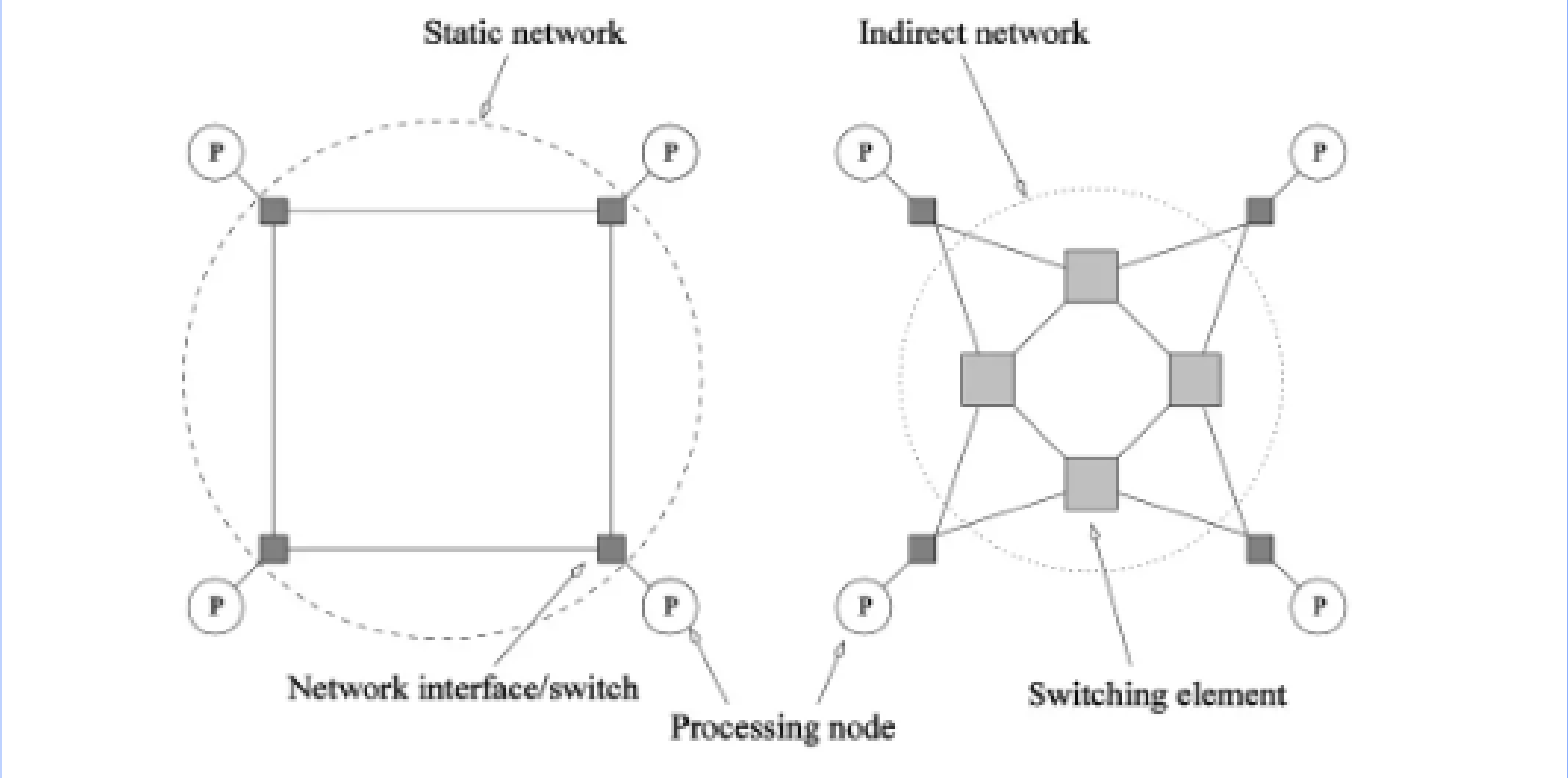
Used to connect processors and memory in parallel systems.

Types

- Static Network – Fixed connections (e.g., Ring Network)
- Dynamic Network – Flexible switching paths (e.g., Omega Network)

Switch Features

- Routing – Path selection for data transfer
- Broadcasting – One processor sends data to all
- Multicasting – One processor sends data to selected processors



COMMUNICATION COSTS IN PARALLEL MACHINES

Basic Idea

- In parallel systems, processors need to exchange data.
- The time required for data transfer between processors is called communication cost.

Components of Communication Cost

- ◆ Latency (Startup Time)
 - Time required to initiate communication.
- ◆ Bandwidth (Transfer Rate)
 - Amount of data transferred per unit time.
- ◆ Overhead
 - Time spent by processors managing communication.

Communication Cost Formula

$$\text{Communication Time} = \text{Latency} + (\text{Message Size} / \text{Bandwidth})$$

Impact

- High communication cost can reduce parallel performance.
- Efficient algorithms try to minimize communication between processors.

LEVELS OF PARALLELISM,

- Parallelism can occur at different levels of a computer system.
- Each level improves performance by executing multiple operations simultaneously.

Main Levels of Parallelism

Bit-Level Parallelism

- Multiple bits processed in one operation
- Example: 64-bit processor processes 64 bits at once

Instruction-Level Parallelism (ILP)

- Multiple instructions executed simultaneously
- Example: Pipelining and superscalar processors

Data-Level Parallelism (DLP)

- Same operation performed on multiple data elements
- Example: Vector processing, SIMD

Task-Level Parallelism (TLP)

- Different tasks executed in parallel by multiple processors or threads



MODELS : SIMD

A parallel computing model where one instruction operates on multiple data elements simultaneously.

Working

- A single control unit sends the same instruction
- Multiple processing units execute the instruction on different data

Example

Add two arrays:

A = [1, 2, 3, 4]

B = [5, 6, 7, 8]

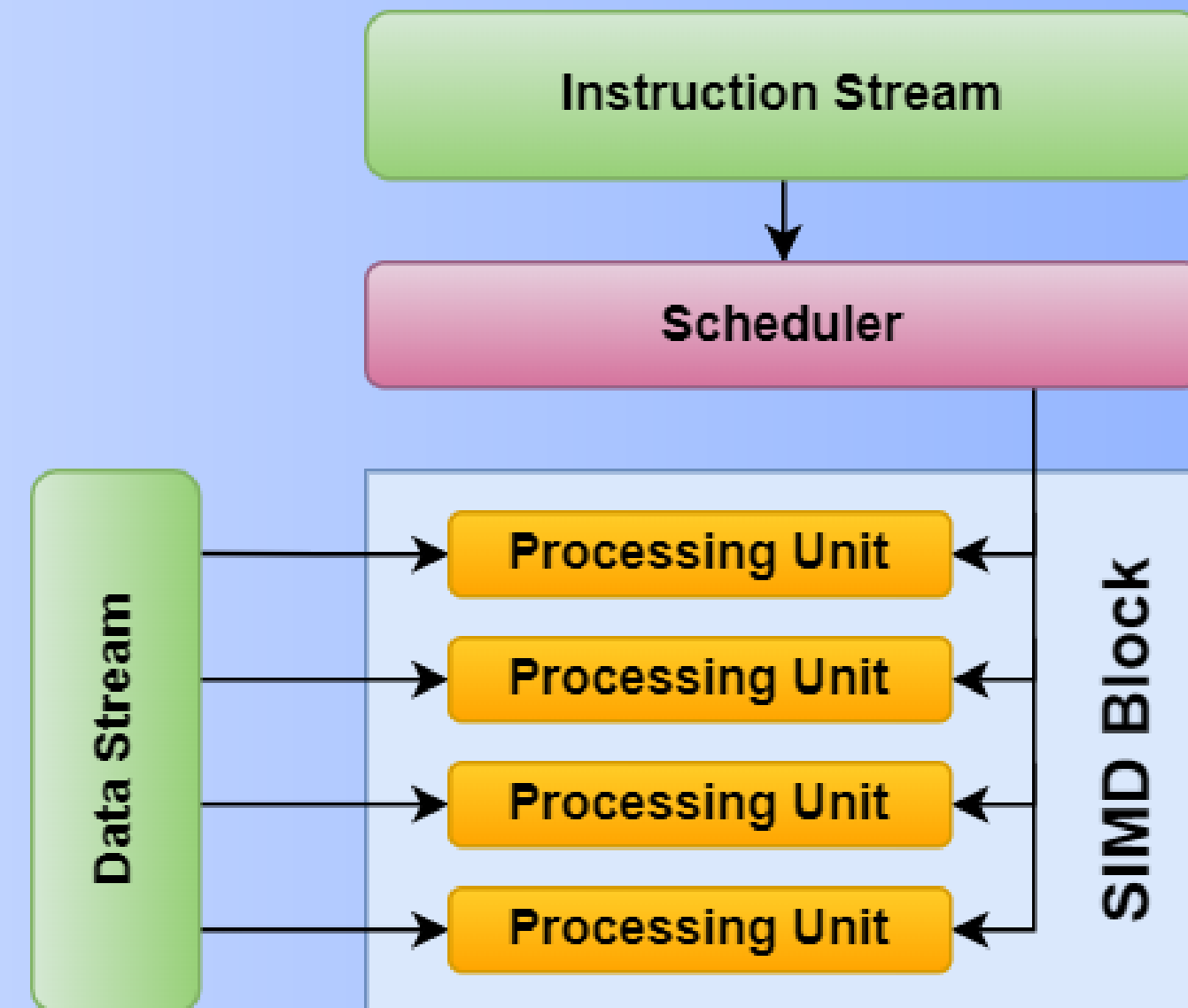
SIMD performs addition of all elements in parallel.

Applications

- Image processing
- Graphics processing
- Scientific computations
- Machine learning

Advantages

- ✓ High performance for data-parallel tasks
- ✓ Efficient use of hardware
- ✓ Faster execution of repetitive operations



MODELS : MIMD

- Parallel computing model where multiple processors execute different instructions on different data simultaneously.

Working

- Each processor has its own control unit
- Processors operate independently
- Different tasks are executed in parallel

Types of MIMD Systems

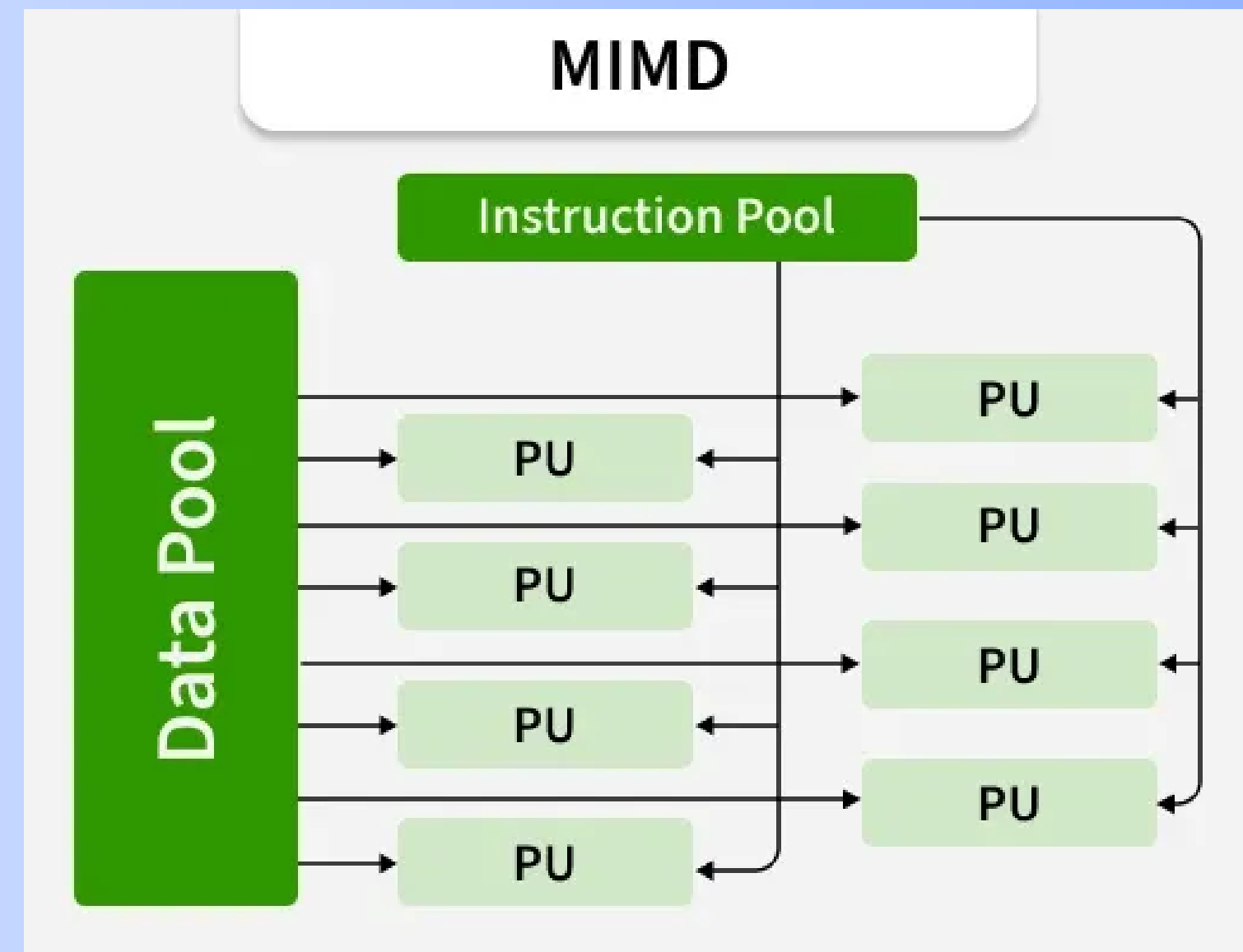
- ◆ Shared Memory Systems
 - Processors share common memory
- ◆ Distributed Memory Systems
 - Each processor has its own local memory

Applications

- Scientific simulations
- Database systems
- Large-scale parallel computing

Advantages

- ✓ High flexibility
- ✓ Efficient task parallelism
- ✓ Suitable for complex applications



MODELS : SIMT

A parallel computing model used mainly in GPU architectures . A single instruction is executed by multiple threads simultaneously

Working

- Threads are grouped into warps
- All threads in a warp execute the same instruction
- Each thread operates on different data

Applications

- Graphics processing
- Image processing
- Deep learning
- Scientific simulations

Advantage

- ✓ Efficient for large-scale data-parallel tasks

MODELS : SPMD

Multiple processors execute the same program .Each processor works on different data

Working

- Same program is loaded on all processors
- Processors follow different execution paths based on data

Applications

- Scientific computing
- Parallel simulations
- High-performance computing

DATA FLOW MODELS

- A computation model where execution of instructions depends on the availability of data.
- Instructions execute as soon as required data is available.

Key Characteristics

- No fixed sequence of instructions
- Instructions represented as nodes in a graph
- Data values move along edges between nodes

Working

1. Program represented as a Data Flow Graph
2. Nodes represent operations
3. Edges represent data dependencies
4. When input data is ready → operation executes

Advantages

- ✓ High parallelism
- ✓ Efficient resource utilization
- ✓ Reduced idle time

DEMAND-DRIVEN COMPUTATION

Computation is performed only when the result is required.

Execution starts from the output request and works backward to find needed inputs.

Working

- Program requests a result (output).
- System checks what data is required to produce that result.
- Required operations are executed only when needed.

Key Characteristics

- Also called Lazy Evaluation
- Avoids unnecessary computations
- Improves efficiency in some applications

Applications

- Functional programming languages
- Query processing systems
- Artificial intelligence applications

Advantages

- ✓ Reduces unnecessary computation
- ✓ Efficient resource utilization
- ✓ Saves processing time

N-WIDE SUPERSCALAR ARCHITECTURES

- A processor architecture that can issue and execute multiple instructions in one clock cycle.
- N-wide means the processor can issue N instructions simultaneously.

Working

- Instructions are fetched and decoded together
- Multiple execution units perform operations in parallel
- Hardware detects instruction-level parallelism (ILP)

Example

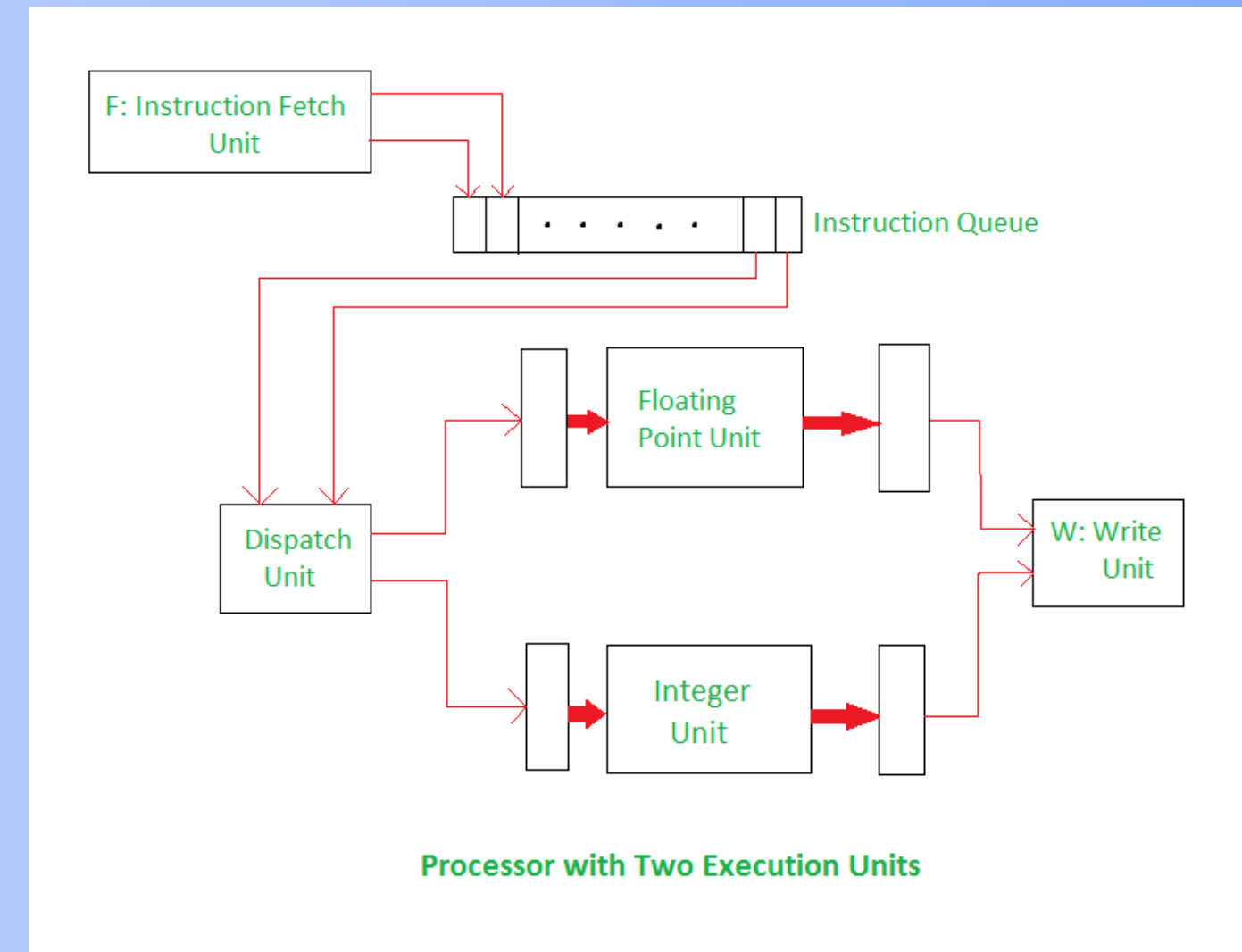
- 2-wide superscalar → executes 2 instructions per cycle
- 4-wide superscalar → executes 4 instructions per cycle

Advantages

- ✓ Increased instruction throughput
- ✓ Better CPU performance
- ✓ Efficient use of execution units

Limitation

- Hardware complexity increases
- Dependency between instructions may reduce parallelism



MULTI-CORE AND MULTI-THREADED PROCESSORS

Multi-Core Processors

- A processor that contains two or more cores on a single chip.
- Each core can execute instructions independently.
- Improves parallel processing and system performance.
- Examples: Dual-core, Quad-core, Octa-core processors.

Advantages

- Higher performance
- Better multitasking
- Lower power consumption compared to multiple CPUs

Multi-Threaded Processors

- A technique where multiple threads run on a single processor core.
- The CPU switches between threads to improve utilization of resources.

Types

- Fine-grained multithreading
- Coarse-grained multithreading
- Simultaneous multithreading (SMT)

Benefits

- Improves CPU utilization
- Reduces idle time
- Enhances overall system efficiency

THANK YOU!



SUBSCRIBE

